VASTRA: Security Architecture & Threat Model

Application Security Report | Component: Cloud Infrastructure & API Interoperability

This document outlines the security posture of the VASTRA platform, an AI-driven e-commerce discovery application utilizing a React frontend, FastAPI ASGI microservices, and Microsoft Azure cloud infrastructure. The threat model is analyzed using the industry-standard STRIDE methodology, detailing specific attack vectors and the implemented architectural mitigations.

1. STRIDE Threat Model Analysis

The system architecture encompasses several trust boundaries, notably between the client browser, the FastAPI backend, Firebase Authentication services, Azure SQL, and external APIs (Oxylabs, Groq). The following matrix classifies primary threats across these boundaries.

Threat Category	Identified Attack Vector	Implemented Mitigation Strategy
Spoofing (Identity)	An attacker attempts to impersonate a legitimate user or access the API by forging session tokens or exploiting weak authentication.	Integration of **Firebase Authentication**. The backend utilizes `dependencies/auth.py` to cryptographically verify Google-signed JWT Bearer tokens on every protected route. Anonymous API access is strictly denied.
Tampering (Data Integrity)	Malicious interception and modification of payload data (e.g., modifying product prices, wishlist IDs, or user roles) in transit.	Enforced **TLS 1.2+** across all Azure App Service endpoints. Database queries utilize SQLAlchemy ORM and parameterized execution (e.g., `sp_UpsertProductReview`) to fundamentally neutralize SQL Injection (SQLi) attempts.
Repudiation (Auditability)	A user performs a destructive or unauthorized action (e.g., mass API scraping) and subsequently denies performing the action.	Implementation of an immutable `activity_logs` relational table in Azure SQL. Critical API interactions are logged within ACID-compliant transaction blocks, securely associating operations with the verified Firebase UID.
Information Disclosure	Unauthorized exposure of external API keys (Oxylabs, Groq), database connection strings, or internal infrastructure topologies.	Complete exclusion of `.env` files from version control via `.gitignore`. Production secrets are isolated within Azure Web App Environment Variables and GitHub Actions Secrets. The Azure Blob container is restricted strictly to read-only access for blob rendering.
Denial of Service (Availability)	Resource exhaustion via repeated, rapid requests to external scraping endpoints (Oxylabs), draining financial credits or backend compute.	The React frontend utilizes `AbortController` to sever orphaned requests. The FastAPI backend wraps the Oxylabs HTTPX client in strict `asyncio.wait_for` timeout parameters, decoupling AI rendering from scraping latency.

Threat Category	Identified Attack Vector	Implemented Mitigation Strategy
Elevation of Privilege	A standard authenticated user attempts to execute administrative functions, such as viewing telemetry or modifying global schemas.	Enforced **Role-Based Access Control (RBAC)**. The JWT payload is interrogated for role claims. The `require_role("admin")` dependency actively rejects standard user access to administrative endpoints with HTTP 403 status codes.

2. Cloud Infrastructure Security Checklist

The following audit checklist verifies the operational security controls applied to the deployment environments, ensuring compliance with DP-900 cloud data principles and general production-readiness standards.

Control Area	Verification Item	Status	Component
Identity & Access (IAM)	**Firebase project configured to restrict sign-ins to authorized domains (Azure Static Web Apps).**	**Verified**	Firebase
	JWT structural integrity and expiration validated on the server side prior to route execution.	**Verified**	FastAPI Auth
	Role privileges mapped accurately in database/token (Admin vs. Standard).	**Verified**	FastAPI RBAC
Data Security (At Rest & Transit)	**Azure SQL Server Firewall restricts inbound traffic strictly to allowed Azure services and specific client IPs.**	**Verified**	Azure SQL
	ODBC Driver configured with `Encrypt=yes` and `TrustServerCertificate=no` in production.	**Verified**	pyodbc
	Azure Blob Storage Account public access explicitly disabled at the account level; container restricted to blob-read only.	**Verified**	Azure Blob
Application & API Security	**Cross-Origin Resource Sharing (CORS) strictly defines allowed frontend origins; wildcard origins (*) disabled.**	**Verified**	FastAPI CORS
	Vite environment variables contain only public SDK keys (Firebase). No private backend keys are bundled into the static JS output.	**Verified**	Vite Build
	Oxylabs Web API integrated using Bearer Token authorization via HTTP headers, preventing credential exposure in URIs.	**Verified**	Oxylabs
DevOps & CI/CD Pipeline	**Git commit history sanitized using `filter-repo`/new repository initialization to guarantee zero historic API key leakage.**	**Verified**	Git VCS

Control Area	Verification Item	Status	Component
Deployment tokens and Azure Publish Profiles secured as encrypted GitHub Action Secrets.	**Verified**	GitHub Actions	

Summary Declaration: The VASTRA platform adheres to secure-by-design principles. Relational data maintains 3NF integrity within an isolated Azure SQL instance, while blob storage strictly segments unstructured media. Authentication pathways effectively prevent horizontal and vertical privilege escalation, rendering the architecture suitable for production-grade e-commerce operations.